

Prompt:

Write a C program that follows these requirements.

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Chat GPT (website):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_SUBJECT 500
#define MAX_CATEGORY 50

typedef struct {
    char sender[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    char date[11];          // MM-DD-YYYY + '\0'
    int priority;           // Higher number = higher priority
    int dateValue;         // Used to compare dates
} Email;

typedef struct {
    Email emails[MAX_EMAILS];
    int size;
} MaxHeap;

/*
 * Convert MM-DD-YYYY into a number that can be compared.
 * YYYYMMDD gives chronological ordering.
 */
int dateToValue(const char *date) {
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

/*
 * Return the priority associated with a sender category.
 */
int getPriority(const char *sender) {
```

```

    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1;    // OtherPerson
}

/*
 * Determine whether email a has higher priority than email b.
 *
 * Sender category is considered first.
 * If both have the same category, the newest date comes first.
 */
int isHigherPriority(Email a, Email b) {
    if (a.priority > b.priority)
        return 1;

    if (a.priority < b.priority)
        return 0;

    /* Same sender category: newest email comes first. */
    if (a.dateValue > b.dateValue)
        return 1;

    return 0;
}

/*
 * Swap two emails.
 */
void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

```

}

/*
 * Move an element upward in the MaxHeap.
 */
void heapifyUp(MaxHeap *heap, int index) {
    int parent;

    while (index > 0) {
        parent = (index - 1) / 2;

        if (isHigherPriority(heap->emails[index],
                             heap->emails[parent])) {
            swapEmails(&heap->emails[index],
                      &heap->emails[parent]);

            index = parent;
        } else {
            break;
        }
    }
}

/*
 * Move an element downward in the MaxHeap.
 */
void heapifyDown(MaxHeap *heap, int index) {
    int left;
    int right;
    int largest;

    while (1) {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        if (left < heap->size &&
            isHigherPriority(heap->emails[left],
                            heap->emails[largest])) {
            swapEmails(&heap->emails[largest],
                      &heap->emails[left]);
            largest = left;
        }
        if (right < heap->size &&
            isHigherPriority(heap->emails[right],
                            heap->emails[largest])) {
            swapEmails(&heap->emails[largest],
                      &heap->emails[right]);
            largest = right;
        }

        index = largest;
    }
}

```

```

        largest = left;
    }

    if (right < heap->size &&
        isHigherPriority(heap->emails[right],
                        heap->emails[largest])) {
        largest = right;
    }

    if (largest != index) {
        swapEmails(&heap->emails[index],
                  &heap->emails[largest]);

        index = largest;
    } else {
        break;
    }
}

}

/*
 * Initialize the MaxHeap.
 */
void initializeHeap(MaxHeap *heap) {
    heap->size = 0;
}

/*
 * Insert an email into the MaxHeap.
 */
void insertEmail(MaxHeap *heap, Email email) {
    if (heap->size >= MAX_EMAILS)
        return;

    heap->emails[heap->size] = email;

    heapifyUp(heap, heap->size);

    heap->size++;
}

```

```

}

/*
 * Return the highest-priority email without removing it.
 */
Email *peekMax(MaxHeap *heap) {
    if (heap->size == 0)
        return NULL;

    return &heap->emails[0];
}

/*
 * Remove the highest-priority email from the MaxHeap.
 *
 * This is used for READ.
 */
void removeMax(MaxHeap *heap) {
    if (heap->size == 0)
        return;

    heap->emails[0] = heap->emails[heap->size - 1];
    heap->size--;

    if (heap->size > 0)
        heapifyDown(heap, 0);
}

/*
 * Read an EMAIL line and add the email to the heap.
 *
 * Expected format:
 * EMAIL <sender>,<subject>,<date>
 */
void processEmail(char *line, MaxHeap *heap) {
    char *firstComma;
    char *secondComma;

    char sender[MAX_CATEGORY];

```

```
char subject[MAX_SUBJECT];
char date[11];

/*
 * Skip "EMAIL ".
 */
char *data = line + 6;

firstComma = strchr(data, ',');

if (firstComma == NULL)
    return;

secondComma = strchr(firstComma + 1, ',');

if (secondComma == NULL)
    return;

/*
 * Extract sender.
 */
*firstComma = '\0';
strcpy(sender, data);

/*
 * Extract subject.
 */
*secondComma = '\0';
strcpy(subject, firstComma + 1);

/*
 * Extract date.
 */
strcpy(date, secondComma + 1);

/*
 * Remove trailing newline from date.
 */
date[strcspn(date, "\r\n")] = '\0';
```

```

    Email email;

    strcpy(email.sender, sender);
    strcpy(email.subject, subject);
    strcpy(email.date, date);

    email.priority = getPriority(sender);
    email.dateValue = dateToValue(date);

    insertEmail(heap, email);
}

/*
 * Process the NEXT command.
 *
 * NEXT only displays the highest-priority email.
 * It does NOT remove the email.
 */
void processNext(MaxHeap *heap) {
    Email *email = peekMax(heap);

    if (email == NULL)
        return;

    printf("Next email:\n");
    printf("Sender: %s\n", email->sender);
    printf("Subject: %s\n", email->subject);
    printf("Date: %s\n", email->date);
}

/*
 * Process the READ command.
 *
 * READ removes the highest-priority email.
 * It does not display anything.
 */
void processRead(MaxHeap *heap) {
    removeMax(heap);
}

```

```

}

/*
 * Process the COUNT command.
 */
void processCount(MaxHeap *heap) {
    printf("There are %d emails to read.\n", heap->size);
}

int main(void) {
    MaxHeap heap;
    char line[700];

    initializeHeap(&heap);

    /*
     * Read commands until EOF.
     */
    while (fgets(line, sizeof(line), stdin) != NULL) {

        /*
         * Remove trailing newline when checking commands.
         */
        if (strncmp(line, "EMAIL ", 6) == 0) {
            processEmail(line, &heap);
        }
        else if (strncmp(line, "NEXT", 4) == 0) {
            processNext(&heap);
        }
        else if (strncmp(line, "READ", 4) == 0) {
            processRead(&heap);
        }
        else if (strncmp(line, "COUNT", 5) == 0) {
            processCount(&heap);
        }
    }

    return 0;
}

```


Claude (website):

```
/* =====
* email_priority.c
*
* Priority-queue-based email triage system for a busy CEO,
* implemented with a hand-rolled, list-based (array-backed) MaxHeap.
*
* Commands read one per line from standard input:
*
* EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
* NEXT
* READ
* COUNT
*
* Priority rules:
* 1) Sender category, highest first:
* Boss > Subordinate > Peer > ImportantPerson > OtherPerson
* 2) Within the same category, the NEWEST date wins (read first).
*
* No pre-existing heap library is used -- every heap operation
* (insert / extract-max / sift-up / sift-down / resize) is written
* from scratch below.
* ===== */

#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* -----
* Email record
* ----- */
#define MAX_SENDER 64
#define MAX_SUBJECT 256
#define MAX_DATE 16

typedef struct {
char sender[MAX_SENDER]; /* the raw category string, e.g. "Boss" */
```

```

char subject[MAX_SUBJECT];
char date[MAX_DATE]; /* MM-DD-YYYY, stored as given */
int categoryRank; /* precomputed numeric priority for speed */
long dateKey; /* YYYYMMDD as an integer for comparisons */
} Email;

/* -----
 * MaxHeap : list-based (dynamic array) implementation
 * ----- */
typedef struct {
    Email *data; /* backing array (the "list") */
    int size; /* number of elements currently stored */
    int capacity; /* allocated capacity of data[] */
} MaxHeap;

/* ---- forward declarations ----- */
static void heapInit(MaxHeap *h, int initialCapacity);
static void heapFree(MaxHeap *h);
static void heapGrowIfNeeded(MaxHeap *h);
static int emailIsHigherPriority(const Email *a, const Email *b);
static void heapSwap(Email *a, Email *b);
static void heapSiftUp(MaxHeap *h, int index);
static void heapSiftDown(MaxHeap *h, int index);
static void heapInsert(MaxHeap *h, Email e);
static int heapExtractMax(MaxHeap *h, Email *out); /* returns 0/1
success */
static int heapPeek(const MaxHeap *h, Email *out); /* returns 0/1
success */

/* -----
 * Category rank: higher number == higher priority
 * ----- */
static int categoryRank(const char *category)
{
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
}

```

```
return 0; /* unknown category: lowest priority, shouldn't occur */
}
```

```
/* Convert "MM-DD-YYYY" into an integer YYYYMMDD so that a later
 * date compares as numerically larger than an earlier one. */
```

```
static long dateToKey(const char *date)
```

```
{
```

```
int mm = 0, dd = 0, yyyy = 0;
```

```
if (sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy) != 3) {
```

```
return 0; /* malformed date -> treat as earliest possible */
```

```
}
```

```
return ((long)yyyy * 10000L) + ((long)mm * 100L) + dd;
```

```
}
```

```
/* =====
```

```
* MaxHeap implementation
```

```
* ===== */
```

```
static void heapInit(MaxHeap *h, int initialCapacity)
```

```
{
```

```
if (initialCapacity < 4) initialCapacity = 4;
```

```
h->data = (Email *)malloc(sizeof(Email) * (size_t)initialCapacity);
```

```
h->size = 0;
```

```
h->capacity = initialCapacity;
```

```
}
```

```
static void heapFree(MaxHeap *h)
```

```
{
```

```
free(h->data);
```

```
h->data = NULL;
```

```
h->size = 0;
```

```
h->capacity = 0;
```

```
}
```

```
static void heapGrowIfNeeded(MaxHeap *h)
```

```
{
```

```
if (h->size >= h->capacity) {
```

```
int newCapacity = h->capacity * 2;
```

```

Email *newData = (Email *)realloc(h->data, sizeof(Email) *
(size_t)newCapacity);
if (newData == NULL) {
fprintf(stderr, "Error: out of memory while growing heap.\n");
exit(EXIT_FAILURE);
}
h->data = newData;
h->capacity = newCapacity;
}
}

/* Returns 1 if email a has strictly higher priority than email b,
* 0 otherwise. This is the single source of truth for ordering. */
static int emailIsHigherPriority(const Email *a, const Email *b)
{
if (a->categoryRank != b->categoryRank) {
return a->categoryRank > b->categoryRank;
}
/* Same category: newer date (larger dateKey) is higher priority */
return a->dateKey > b->dateKey;
}

static void heapSwap(Email *a, Email *b)
{
Email tmp = *a;
*a = *b;
*b = tmp;
}

/* Move the element at index up until heap property is restored. */
static void heapSiftUp(MaxHeap *h, int index)
{
while (index > 0) {
int parent = (index - 1) / 2;
if (emailIsHigherPriority(&h->data[index], &h->data[parent])) {
heapSwap(&h->data[index], &h->data[parent]);
index = parent;
} else {
break;
}
}
}

```

```

}
}
}

/* Move the element at index down until heap property is restored. */
static void heapSiftDown(MaxHeap *h, int index)
{
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < h->size && emailIsHigherPriority(&h->data[left], &h->data[largest])) {
            largest = left;
        }
        if (right < h->size && emailIsHigherPriority(&h->data[right], &h->data[largest])) {
            largest = right;
        }
        if (largest == index) break;

        heapSwap(&h->data[index], &h->data[largest]);
        index = largest;
    }
}

static void heapInsert(MaxHeap *h, Email e)
{
    heapGrowIfNeeded(h);
    h->data[h->size] = e;
    heapSiftUp(h, h->size);
    h->size++;
}

/* Removes and returns (via out) the highest-priority email.
 * Returns 1 on success, 0 if the heap was empty. */
static int heapExtractMax(MaxHeap *h, Email *out)
{

```

```

if (h->size == 0) return 0;

if (out != NULL) {
*out = h->data[0];
}

h->size--;
if (h->size > 0) {
h->data[0] = h->data[h->size];
heapSiftDown(h, 0);
}
return 1;
}

/* Looks at (without removing) the highest-priority email.
 * Returns 1 on success, 0 if the heap was empty. */
static int heapPeek(const MaxHeap *h, Email *out)
{
if (h->size == 0) return 0;
if (out != NULL) {
*out = h->data[0];
}
return 1;
}

/* =====
 * Command parsing helpers
 * ===== */

/* Strip trailing '\n', '\r', and any trailing whitespace in-place. */
static void trimTrailing(char *s)
{
size_t len = strlen(s);
while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r' ||
isspace((unsigned char)s[len - 1]))) {
s[len - 1] = '\0';
len--;
}
}

```

```

/* Parse a line of the form:
 * EMAIL <category>,<subject>,<date>
 * (the line passed in has already had the leading "EMAIL " removed)
 * Fields are comma-delimited; subject may contain spaces but not
 * commas.
 * Returns 1 on success, 0 on malformed input. */
static int parseEmailFields(char *rest, Email *e)
{
    /* rest looks like: "Peer,Can you help me on this?,12-01-2024" */
    char *category = strtok(rest, ",");
    char *subject = strtok(NULL, ",");
    char *date = strtok(NULL, ",");

    if (category == NULL || subject == NULL || date == NULL) {
        return 0;
    }

    /* Category may have stray surrounding whitespace; trim it. */
    while (isspace((unsigned char)*category)) category++;

    /* Subject: trim a single leading space if present (after the comma),
     * but preserve internal spaces since subjects may contain them. */
    if (*subject == ' ') subject++;

    if (*date == ' ') date++;
    trimTrailing(date);

    strncpy(e->sender, category, MAX_SENDER - 1);
    e->sender[MAX_SENDER - 1] = '\0';

    strncpy(e->subject, subject, MAX_SUBJECT - 1);
    e->subject[MAX_SUBJECT - 1] = '\0';

    strncpy(e->date, date, MAX_DATE - 1);
    e->date[MAX_DATE - 1] = '\0';

    e->categoryRank = categoryRank(e->sender);
    e->dateKey = dateToKey(e->date);

```

```

return 1;
}

/* =====
* Main driver
* ===== */
int main(void)
{
    MaxHeap heap;
    heapInit(&heap, 16);

    char *line = NULL;
    size_t lineCap = 0;
    ssize_t nread;

    while ((nread = getline(&line, &lineCap, stdin)) != -1) {
        (void)nread;
        trimTrailing(line);

        if (line[0] == '\\0') {
            continue; /* skip blank lines */
        }

        if (strncmp(line, "EMAIL ", 6) == 0) {
            char *rest = line + 6;
            Email e;
            if (parseEmailFields(rest, &e)) {
                heapInsert(&heap, e);
            } else {
                fprintf(stderr, "Warning: malformed EMAIL line ignored: %s\\n", line);
            }
        }
        else if (strcmp(line, "NEXT") == 0) {
            Email top;
            if (heapPeek(&heap, &top)) {
                printf("Next email:\\n");
                printf("Sender: %s\\n", top.sender);
                printf("Subject: %s\\n", top.subject);
            }
        }
    }
}

```

```
printf("Date: %s\n", top.date);
} else {
printf("There are no emails to read.\n");
}
}
else if (strcmp(line, "READ") == 0) {
/* Removes the highest-priority email, if any, without
 * displaying anything -- matches spec for both the
 * normal case and two READs in a row with no NEXT. */
heapExtractMax(&heap, NULL);
}
else if (strcmp(line, "COUNT") == 0) {
printf("There are %d emails to read.\n", heap.size);
}
else {
fprintf(stderr, "Warning: unrecognized command ignored: %s\n", line);
}
}

free(line);
heapFree(&heap);
return 0;
}
```

Analysis:

1) Correctness:

a) ChatGPT

1. Code executes but does not give correct output, sender is blank and the order is not correct
2. Does not handle edge cases or missing inputs
3. Has no meaningful error messages, but still runs with incorrect input

b) Claude

1. Code executes with mostly outputs
2. Handles incorrect inputs by skipping over the incorrect input
3. Has meaningful error messages and still runs with incorrect input

2) Time Complexity:

a) ChatGPT

1. The time complexity for the code is $O(n)$ as the max heap should have complexity $O(\log n)$ and the while loop at the beginning depends on input size.

b) Claude

1. Same time complexity as the code from ChatGPT, although they likely have different execution time it is unnecessary to test as ChatGPT failed the correctness test and will not be after the comparison

3) Space Complexity

a) Comparison

1. Both programs use two structs with 5 and 2 variables respectively, neither uses recursive calls, They both use a list for the max heap. One notable difference though is that when defining the max lengths for the email struct ChatGPT uses much larger values than Claude

4) Maintainability

a) ChatGPT

1. Has comments with a limited explanation on what each block of code does, but no explanation on how or why

b) Claude

1. Has comments that do a much better job explaining what a block of code is doing both on how it is doing it and why.

5) Final Decision

- a) Due to better or equal scores in every category, I will be using the code written by Claude.

6) Changes

- a) I added better maintainability by both modifying and adding comments on every line of code.
- b) I also fixed the outputs in the main function by adding “\t” to indent the outputs after the “Next Email:”.